

How I implemented the update profile

why route, middleware, controller? also explain the flow in simple way , i dont get it

This is actually **the most important concept in Express.js**, and once you understand it, every backend project will make sense.

Let's use a **real-life analogy** first.

Imagine a Restaurant 🍽️

Suppose you go to a restaurant.

You say:

I want a Veg Burger.

Does the chef hear you directly?

No.

There are several people involved.

```
Customer
  ↓
Receptionist
  ↓
Security Check (if needed)
  ↓
Chef
  ↓
Food
  ↓
Customer
```

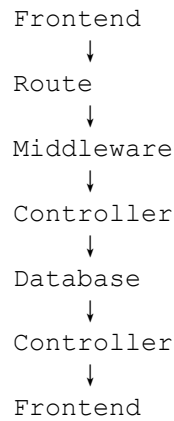
Backend works exactly like this.

In Express

Suppose React sends

```
PUT /api/profile
```

This request goes through



Now let's understand each one.

1. Route (Receptionist)

Think of Route as the receptionist.

The receptionist doesn't cook food.

She only says

"Oh, Profile update? Go to Room 3."

Similarly,

```
router.put("/", authMiddleware, updateProfile);
```

means

"When someone calls

```
PUT /api/profile
```

first execute

```
authMiddleware
```

then

```
updateProfile
```

That's all.

Route **doesn't update the database.**

It only decides

Which function should run?

Example

Imagine

`/api/login`

`/api/register`

`/api/profile`

`/api/products`

`/api/orders`

Route simply directs requests.

Login

↓

`loginController()`

Register

↓

`registerController()`

Profile

↓

`profileController()`

Products

↓

`productController()`

2. Middleware (Security Guard)

Now suppose you enter a company.

Before going inside,

Security asks

Show your ID Card.

If valid,

you can enter.

Otherwise,

No Entry.

JWT Middleware is exactly that.

React sends

Authorization

Bearer eyJhbGc...

Middleware checks

`jwt.verify(token)`

If correct

Go Ahead

If wrong

401 Unauthorized

The controller never runs.

So middleware acts like

Route

↓

Security Guard

↓

Controller

Why Middleware?

Imagine without middleware.

Every controller would have

Check Token

↓

Verify Token

↓

Extract User ID

again

again

again

for every API.

That's repetitive.

Instead

we make one middleware

`authMiddleware`

Now every protected API can reuse it.

Much cleaner.

3. Controller (Chef)

Now security allows you inside.

Who actually cooks?

The chef.

Controller is the chef.

Controller contains the business logic.

Example

```
const user =  
await User.findById(req.user.id);
```

Controller is saying

Go to database

Find this user.

Then

```
user.phone=phone;
```

Change phone.

Then

```
await user.save();
```

Save it.

Finally

```
res.json(...)
```

Send response.

Controller does the real work.

4. Model (Database Manager)

Controller doesn't know MongoDB syntax.

Instead,

Controller asks Model.

```
User.findById(...)
```

User here is the model.

Model knows

How to communicate with MongoDB.

Complete Simple Flow

Suppose you click

Update Profile

React executes

```
axios.put("/api/profile")
```

Now see the flow.

Step 1

React

```
PUT /api/profile
```

↓

Backend receives request.

Step 2

Express looks at routes.

```
router.put("/",  
  authMiddleware,  
  updateProfile)
```

Express says

Oh!

User wants profile update.

Run

```
authMiddleware
```

Step 3

Middleware checks token.

Token valid?

↓

Yes

Store

`req.user.id`

Then

`next();`

What is `next()`?

It means

Continue to next function.

Next function is

`updateProfile()`

Step 4

Controller starts.

`updateProfile()`

Controller asks MongoDB

`User.findById(...)`

MongoDB returns

Ishita

Controller changes

`user.phone`

`user.name`


```
user.shopCategory
```

Then

```
user.save();
```

MongoDB updated.

Step 5

Controller sends response.

```
res.json({  
  
  success:true  
})
```

Response goes back.

Step 6

React receives response.

Shows

Profile Updated Successfully

Done.

Whole Flow Diagram

You click Update



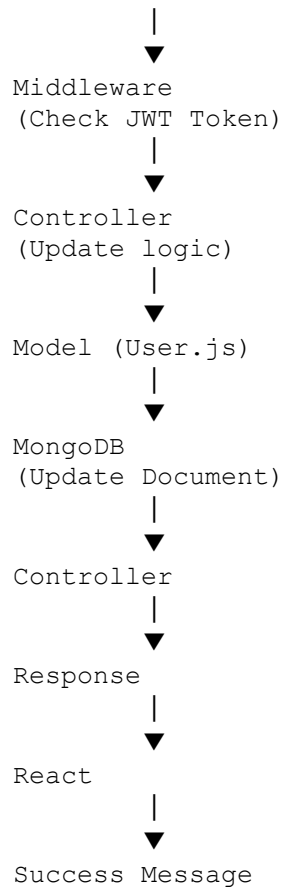
React (Axios)



PUT /api/profile



Route



One Line Definitions (Interview)

Route

"Routes define the API endpoints and decide which function should handle a request."

Example:

```
router.put("/", authMiddleware, updateProfile);
```

Middleware

"Middleware executes before the controller. It performs common tasks like authentication, logging, or validation."

Example:

```
authMiddleware
```

Controller

"Controllers contain the business logic. They process the request, interact with the database using models, and send the response."

Example:

```
updateProfile()
```

Model

"A model defines the structure of the MongoDB collection and provides methods to interact with the database."

Example:

```
User.findById()
```

An easy way to remember

Think of it as **Google Maps**:

- **Route** = Decides **where** the request should go.
- **Middleware** = Checks **whether** the request is allowed to proceed.
- **Controller** = Decides **what** work needs to be done.
- **Model** = Knows **how** to read from and write to the database.
- **Database** = Stores the actual data.

This separation keeps the code organized, reusable, and easier to maintain. For example, the same `authMiddleware` can protect dozens of APIs without duplicating authentication code, and the same `User` model can be reused by different controllers.

In details:

Step 1 Database (MongoDB)

Where?

```
backend-node
├── src
│   └── models
│       └── User.js
```

This file defines how a user document is stored in MongoDB.

Example:

```
const userSchema = new mongoose.Schema({
  name:String,
  email:String,
  password:String,
  phone:String,
  role:String,
  shopCategory:String
});
```

Think of this as a table in SQL.

SQL

| id | name | email | phone | shopCategory |

MongoDB

```
{
  "_id":"6874ab",
  "name":"Ishita",
  "email":"abc@gmail.com",
  "phone":"9432",
  "shopCategory":"dairy"
}
```

When you added

```
shopCategory:{
  type:String,
  enum:["vegetable","dairy","both"],
```

```
    default: "vegetable"  
  }  
}
```

you changed the database schema.

Now every user document can store

Vegetable

or

Dairy

or

Both

Step 2 Login

User enters

Email

Password

React sends

POST

/api/auth/login

Backend receives it.

Inside

authController.js

it checks

```
const user = await User.findOne({email});
```

MongoDB searches

users collection

↓

email matches?

↓

Yes

↓

```
return user
```

Then password is compared

```
bcrypt.compare(password, user.password)
```

Because password inside database is encrypted.

If correct

Backend creates

```
const token = jwt.sign({  
  id:user._id,  
  role:user.role  
});
```

Example token

```
eyJhbGciOiJIUzI1NiIsInR5...
```

React stores it

```
localStorage.setItem("token", token);
```

Now user is authenticated.

Why JWT?

Imagine a website.

User logs in.

Backend says

"I verified you."

Instead of checking username/password on every request, backend gives a **token**.

Later

Update Profile

React sends

Authorization

Bearer eyJhbGciOiJIUz...

Backend simply verifies the token.

Much faster.

Step 3 Profile API

You created

```
profileRoutes.js
router.get("/",
  authMiddleware,
  getProfile);
```

Meaning

Whenever

GET

/api/profile

comes

Execute

authMiddleware

↓

getProfile

Similarly

```
router.put("/",
  authMiddleware,
  updateProfile);
```

Step 4 Middleware

This is one of the favourite interview questions.

Middleware runs **before** controller.

Request



Middleware



Controller



Response

Your middleware

```
const authHeader=req.headers.authorization;
```

Reads

Authorization

Bearer xxxxxxxxxx

Then

```
jwt.verify(token)
```

checks

Is token valid?

If valid

```
req.user=decoded;
```

Suppose token contains

```
{  
  "id":"68483",  
  "role":"user"  
}
```

Now every controller automatically gets

```
req.user.id
```


No need to send user id from frontend.

This is secure.

Step 5 Controller

Your controller

```
const user=await User.findById(req.user.id);
```

Uses the id from JWT.

Then

```
user.name=name;
```

```
user.phone=phone;
```

```
user.shopCategory=shopCategory;
```

Then

```
await user.save();
```

Mongoose updates MongoDB.

Finally

```
res.json({  
  
  success:true  
  
});
```

Response goes back.

Step 6 Frontend

Inside

```
Profile.jsx
```

React loads.

Immediately

```
useEffect(()=>{  
  fetchProfile();  
},[]);
```

Meaning

Page Opens

↓

Fetch profile automatically

Axios

```
axios.get("/api/profile")
```

Header

Authorization:

Bearer token

Backend returns

```
{  
  "name":"Ishita",  
  "phone":"9432"  
}
```

React stores

```
setFormData(response.data.user)
```

Now

Textbox displays

Ishita

9432

Dairy

Step 7 Updating

User changes

Phone

↓

9999999999

Clicks

Update

React executes

```
axios.put(  
  "/api/profile",  
  formData  
)
```

Backend

↓

Middleware

↓

Controller

↓

MongoDB

↓

Updated

↓

Response

↓

React

↓

Alert

Profile Updated Successfully

Complete Flow

User



Login Page



POST /api/auth/login



authController



MongoDB



JWT Generated



Stored in LocalStorage



Open Profile Page



GET /api/profile



JWT Middleware



Profile Controller



MongoDB



User Details



React Form



User edits fields



PUT /api/profile



Middleware



Controller



User Model



MongoDB Updated



Success Response



React shows success

Folder Explanation

backend-node

src



├── models

| User.js

| Database Schema

| (MongoDB Collection Structure)

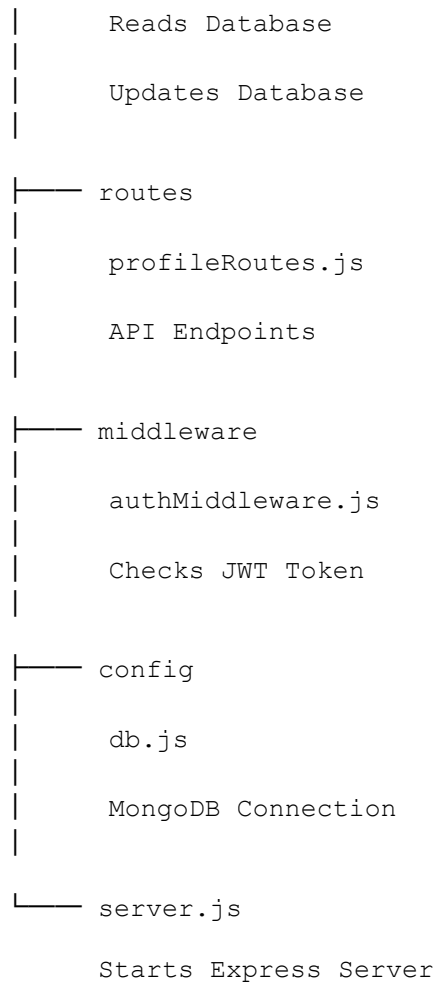


├── controllers

| profileController.js

| Business Logic





React Folder

frontend-react

src

components

shopkeeper

Profile

Profile.jsx

↓

React Component

↓

Calls Backend APIs

↓

Displays Data



Updates Data

If the interviewer asks: "What did you implement?"

You can answer:

"I implemented the Shopkeeper Profile module in our MERN application. First, I extended the MongoDB User schema by adding a shopCategory field with allowed values vegetable, dairy, and both. Then I created protected profile APIs (GET /api/profile and PUT /api/profile) using Express routes. I added a JWT authentication middleware that verifies the token and attaches the logged-in user's ID to req.user, so the frontend never needs to send the user ID explicitly. In the controller, I fetch the user with User.findById(req.user.id), update only the editable fields, and save the document using Mongoose. On the React side, I used useEffect to fetch the profile when the page loads and Axios to communicate with the backend, sending the JWT token in the Authorization: Bearer <token> header. The profile form is pre-filled with the existing data, and when the user updates it, a PUT request updates MongoDB and the UI reflects the changes."